TECHNICAL SPECIFICATION

Food Group Log

A single-page web app for logging which of 13 food groups you eat each day, with a calendar, analytics and backups. No login, no backend. All data is kept in the browser's localStorage.

Written as a handoff for another developer or Claude Code session. It describes the final state of the app as of 29 October 2026, the reasons behind each decision, and how to change it safely.

1. Summary and hard constraints
2. Tech stack and how to run it
3. File structure and load order
4. Data model and storage
5. The 13 food items
6. Architecture and module APIs
7. App shell, navigation and routing
8. Today view
9. Item tiles (day editor)
10. Calendar view and day modal
11. Analytics view
12. Settings view (backup, import, reset)
13. Design system
14. Responsive behavior
15. Accessibility
16. Decisions made with the user (change log)
17. Testing checklist
18. Deploying
19. How to make common changes
20. Known limitations

1. Summary and hard constraints

- **Purpose:** each day, the user checks off which food groups they ate. They can log or edit any past or future date, see a calendar of what was logged, and look at statistics over a date range.
- **No accounts:** no login, sign-up, authentication or server. The app opens directly into the Today view.
- **Persistence:** everything is saved in `localStorage`, so it survives refreshes and reopening the browser. Changes save immediately; there is no Save button.
- **Required storage format** (from the user's brief, do not change): `{ "2026-10-29": ["milk", "fruit", "chicken"], ... }`
- **Static site:** deployable as-is to Netlify, Vercel or GitHub Pages, with no build step.
- **Works on desktop and mobile**, in light and dark mode.
- **Brand palette (required):** white `#FFFFFF` background and cards; primary green `#006600` for buttons, selected states and accents; deep green-black `#001A00` for the header, headings and main text; light green tints such as `#E6F2E6` for subtle backgrounds and hover.

2. Tech stack and how to run it

- **Plain HTML, CSS and JavaScript.** No framework and no bundler. The brief allowed React + Vite + Tailwind *or* plain JS; plain JS was chosen because Node was not installed on the machine, and because it removes the build step.
- **Classic scripts, not ES modules.** Each file is an IIFE that attaches to a global namespace `window.FT`. This lets the app run straight from `file://` (ES modules are blocked there). **Script order in index.html matters**

(section 3).

- **Chart.js 4.4.1** (UMD build), loaded from `https://cdn.jsdelivr.net/npm/chart.js@4.4.1/dist/chart.umd.js`. It is the only third-party script. If it fails to load, chart cards show "Charts could not load. Check your internet connection and reload."; everything else keeps working.
- **Font:** Manrope (weights 400–800) from Google Fonts, with fallback `system-ui, -apple-system, 'Segoe UI', Roboto, sans-serif`.
- **Icons:** inline SVG strings in `js/icons.js`, 24×24 grid, stroke 1.75, round caps and joins, `stroke="currentColor"`. Most paths come from Lucide (ISC licence); *yogurt*, *bread* and *wing* are custom.

Run locally

```
open index.html                       # works directly from disk
python3 -m http.server 5173           # or serve the folder, then visit http://localhost:5173
```

The project also includes `.claude/launch.json`, which defines a server named `food-group-log` (python3 http.server on port 5173) for Claude Code's preview browser. Python's simple server lets the browser cache CSS, so add a cache-busting query (`?v=2`) or hard-reload after editing CSS.

3. File structure and load order

File	Responsibility
`index.html`	Shell: top bar with brand, four tab links and theme toggle; four empty `<section class="view">` containers; toast root; script tags. A small inline script in `<head>` applies a saved light/dark theme before first paint so the page doesn't flash. Comment markers `BUILD:HEAD-START/END` and `BUILD:BODY-START/END` let a script inline everything into one file (section 18).
`css/styles.css`	All styles: design tokens (light, plus dark twice), base styles, components, views and responsive rules. About 620 lines.
`js/core.js`	Creates `window.FT`, `FT.views = {}`, the event bus `FT.bus`, `FT.esc()` (HTML escaping), `FT.toast()`, and `FT.token(name)`, which reads a CSS custom property.
`js/icons.js`	Icon path dictionary; `FT.icon(name, size=20, cls='')` returns an SVG string. `FT.hydrateIcons(root)` fills `<span data-icon="...">` placeholders.
`js/items.js`	`FT.ITEMS` (the 13 items), `FT.ITEM_IDS`, `FT.itemById`, and `FT.normalizeItem()`, which turns a name or id into a stored id, with aliases.
`js/dates.js`	`FT.dates`: local-midnight date helpers, plus formatting with the fixed `en-US` locale.
`js/store.js`	`FT.store`: reads and writes localStorage, cleans data, notifies subscribers, holds settings, and syncs between open tabs.
`js/components/modal.js`	`FT.modal.open / choose / confirm`: in-page dialogs (centered on desktop, a bottom sheet on phones), with a focus trap and Escape to close.
`js/components/day-editor.js`	`FT.dayEditor(container, dateKey, opts)`: the toolbar plus the 13 selectable item tiles for one date.

`js/components/day-modal.js`	`FT.openDay(dateKey)`: a modal containing a day editor, with previous/next day arrows.
`js/views/today.js`	`FT.views.today`
`js/views/calendar.js`	`FT.views.calendar` (also defines `FT.level(count)`, used to shade days)
`js/views/analytics.js`	`FT.views.analytics`
`js/views/settings.js`	`FT.views.preferences` (also defines `FT.loadSampleData()`)
`js/app.js`	Routing between tabs, theme handling, `FT.app.go(name)`, and initialization. Must load last.
`README.md`	User-facing readme: features, how to run and deploy, data format.

Required script order: chart.js (CDN) → core → icons → items → dates → store → modal → day-editor → day-modal → today → calendar → analytics → settings → app.

4. Data model and storage

4.1 Entries

localStorage key `foodGroupLog.entries` holds JSON: an object whose keys are local dates in the form `YYYY-MM-DD`, and whose values are arrays of item ids.

```
{
  "2026-10-28": ["milk", "yogurt", "vegetables", "fruit", "red_meat", "complex_carbs"],
  "2026-10-29": ["milk", "vegetables", "fruit"]
}
```

Rules, enforced by `sanitize()` / `cleanIds()` in store.js every time data is loaded, set, merged or imported:

- A key is kept only if `FT.dates.isKey(k)` is true: it matches `/^\d{4}-\d{2}-\d{2}$/` and is a real calendar date (it round-trips through `Date`).
- Each value goes through `FT.normalizeItem`. Unknown values are dropped, duplicates are removed, and ids are put in canonical `FT.ITEMS` order.
- **Days with no items are deleted**, never stored as `[]`.
- **Older data:** `simple_carbs` is an alias for `bread`, so entries saved before that change load as Bread.
- All dates are **local**, never UTC. Always build keys with `FT.dates.key(date)`, never with `toISOString()`.

4.2 Settings

localStorage key `foodGroupLog.settings`: `{ "theme": "system" | "light" | "dark", "weekStart": 0 | 1 }`. The defaults are `{ theme: 'system', weekStart: 0 }` (0 = Sunday, 1 = Monday).

4.3 Robustness

- Every localStorage read and write is wrapped in try/catch. If the read fails, the app starts with an empty object. If a write fails (storage full or blocked), the user sees the toast "Could not save. Browser storage may be full or blocked."
- **Sync between tabs:** a `storage` event listener reloads entries or settings when another tab changes them, then notifies subscribers.

5. The 13 food items

Defined in `js/items.js`. The array order is the canonical order, used everywhere (tile grid, stored arrays, CSV columns). **Each item's color is used everywhere that item appears:** its tile icon, the donut chart, the bar chart, the history chips and the summary card icon.

#	id (stored)	Name	Icon key	Color	Icon meaning
1	`milk`	Milk	milk	#3F7CB0	Milk bottle (Lucide)
2	`yogurt`	Yogurt	yogurt	#8661B8	Cup with dollop (custom)
3	`vegetables`	Vegetables	carrot	#2E8B3A	Carrot (Lucide)
4	`fruit`	Fruit	apple	#CC3F3F	Apple (Lucide)
5	`red_meat`	Red Meat	beef	#7A2430	Steak (Lucide beef)
6	`chicken`	Chicken	drumstick	#C77A1E	Drumstick (Lucide)
7	`chicken_wings`	Chicken Wings	wing	#A9472A	Wing with scalloped feathers (custom)
8	`fish`	Fish	fish	#1F8496	Fish (Lucide)
9	`beans`	Beans	bean	#7E6236	Bean (Lucide)
10	`sugar`	Sugar	candy	#D0518A	Wrapped candy (Lucide)
11	`bread`	Bread	bread	#A8861A	Bread slice with crumbs (custom)
12	`complex_carbs`	Complex Carbs	wheat	#5F7F1F	Wheat stalk (Lucide)
13	`processed_food`	Processed Food	package	#5E6672	Package box (Lucide)

Custom icon paths (24×24, stroke-based)

```
yogurt: <path d="M4 9h16"/><path d="M5 9l1.6 11.1A1 1 0 0 0 7.6 21h8.8a1 1 0 0 0 1-.9L19 9"/><path
d="M8 9a4 4 0 0 1 8 0"/>
        <path d="M11.5 5.1c.4-1.4 1.4-2.1 2.5-2.1"/><path d="M7 14h10"/>
bread:  <path d="M7 21h10a1 1 0 0 0 1-1v-8.4A4 4 0 0 0 17 4H7a4 4 0 0 0-1 7.6V20a1 1 0 0 0 1 1Z"/>
        <path d="M10 12v.01"/><path d="M14 14v.01"/><path d="M11 16.5v.01"/>
wing:   <path d="M5 5.5C10 3.8 16.5 5 21 9a3 3 0 0 1-4 3 3 3 0 0 1-4 2 3 3 0 0 1-4 1.5c-3.5 0-5.5-
2.5-5.5-5.5
        0-2.2.6-3.8 1.5-4.5Z"/><path d="m17 12-3-3"/><path d="m13 14-2.5-3"/>
```

Import aliases (`FT.normalizeItem`): the input is lowercased and every run of non-letters becomes `_` (so "Red Meat", "red-meat" and "red_meat" all become `red_meat`). Extra aliases: `processed→processed_food`, `vegetable/veggies→vegetables`, `bean→beans`, `meat→red_meat`, `wings→chicken_wings`, `simple_carbs→bread`.

Item count: the code always reads the count from `FT.ITEMS.length` (13); nothing hard-codes 12 or 13. Badges read like "5/13".

6. Architecture and module APIs

6.1 Event bus (`FT.bus`)

Event	Payload	Emitted by	Listened to by
`settings`	settings object	`store.setSetting`, storage event	app (re-applies theme), calendar, analytics, settings
`theme`	isDark (bool)	app `applyTheme()`	analytics (redraws charts with new colors)
`dayfocus`	dateKey	day modal (on open and on each prev/next)	calendar (moves to that month and selects the day)

Changes to entries don't go through the bus; they use `FT.store.subscribe(fn)`.

6.2 Store API (`FT.store`)

```
get(key)          → string[] copy of ids for that date ([] if none)
count(key)        → number of items on that date
has(key, id)      → boolean
all()             → deep copy of all entries
keys()            → sorted array of logged date keys (cached until the next change)
size()            → number of logged days
set(key, ids)     → cleans the ids, saves (deletes the key if empty), notifies
toggle(key, id)   → adds or removes one id
replaceAll(obj)   → cleans obj and replaces everything
merge(obj)        → per-day union with what's already there (nothing logged is removed)
clear()           → deletes all entries
sanitize(obj)     → returns cleaned entries (used by import)
subscribe(fn)     → calls fn after every change; returns an unsubscribe function
settings()        → copy of the settings
setSetting(k, v)  → saves the setting and emits 'settings'
bytes()           → length of the stored entries string (shown as KB)
```

6.3 Dates API (`FT.dates`)

```
MONTHS, key(d), parse(key), isKey(k), today(), todayKey(), addDays(d,n), addMonths(d,n),
startOfWeek(d, weekStart), startOfMonth(d), endOfMonth(d), diffDays(a,b) (rounded, so DST shifts
are safe),
eachDay(a,b) → Date[], format(d, opts) (en-US), long(d) "Thursday, October 29, 2026",
medium(d) "Oct 29, 2026", weekdayNames(weekStart, 'short'|'long'), relative(d)
"Today|Yesterday|Tomorrow|weekday"
```

Every Date is local midnight. When date math needs a new day, it builds it with `new Date(y, m, d + n)`.

6.4 Components

```
FT.modal.open({ content: string|Node, size: ''|'sm', label, onClose(result) }) → { el,
close(result) }
FT.modal.choose({ title, message(HTML), options:[{label, value, kind:'primary'|'danger'|'',
autofocus}] }) → Promise<value|null>
FT.modal.confirm({ title, message, confirmText, danger }) → Promise<boolean>
FT.dayEditor(container, dateKey, { wide }) → { date (getter), setDate(key), destroy() }
FT.openDay(dateKey)                 // opens the day modal
FT.toast(message)                   // shown for 2.6 s, bottom center (above the mobile tab bar)
FT.level(count) → 0..4              // 0 if count is 0, else min(4, ceil(count / 13 × 4))
```

> Native `alert` / `confirm` / `prompt` are never used. All confirmations are in-page modals, so they also work in sandboxed iframes, such as the claude.ai Artifact preview.

6.5 View interface

Each view object has `show(sectionEl)`. The first call builds the DOM; every call re-renders. Views subscribe to the store once, when they are first built. Analytics redraws on store, settings or theme changes only while its section is visible (`!root.hidden`). Today also has `refresh()`, which app.js calls on `visibilitychange` so an open app moves on to the new day after midnight.

7. App shell, navigation and routing

- **Top bar** (sticky, background `--nav-bg` #001A00, 64px tall; 56px on phones): brand mark (a green rounded square with a leaf icon) and the name "Food Group Log" on the left; tabs **Today / Calendar / Analytics / Settings**, each with an icon (home, calendar, chart, sliders); a theme toggle icon button on the right (moon in light mode, sun in dark mode). The active tab is filled with #006600 and has `aria-current="page"`.
- **Routing** uses URL hashes: `#today`, `#calendar`, `#analytics`, `#preferences`. An unknown or empty hash shows Today. The Settings tab deliberately uses the hash `#preferences`, not `#settings`, because the claude.ai Artifact viewer reserves the word "settings".
- `FT.app.go(name)` updates the hash with `history.replaceState` (wrapped in try/catch for sandboxes) and shows the view. A delegated click handler sends every `<a href="#view">` through `go()`. The `hashchange` event is handled too.
- Switching views toggles each `section.view`'s `hidden` attribute, calls `show()`, and scrolls to the top. The view slides up 6px over 0.25s, a transform only. Opacity is never animated, so a backgrounded tab is never left looking faded.
- **Theme:** the `theme` setting is "system" (follows `prefers-color-scheme`), "light" or "dark". For light or dark, app.js sets `data-theme` on `<html>`; for system it removes the attribute, but only if the app set it. The toggle button switches to whichever theme isn't showing. A matchMedia listener re-applies the theme when the OS setting changes in system mode.

8. Today view

- **Page head:** the eyebrow "TODAY" in small green capitals; an H1 with the full date, e.g. "Thursday, October 29"; on the right, a streak pill (flame icon and "N-day streak", hidden when the streak is 0) and a progress block: "**4**/13 food groups" over an 8px green progress bar (`role="progressbar"`, aria-valuenow).
- **Card:** one day editor for today, with `{ wide: true }`: 7 tiles per row at ≥960px, so 7 + 6.

- **Streak:** the number of consecutive days with at least one item, counting back from today. If today has nothing logged yet, the count starts from yesterday, so the streak isn't lost before you log today.
- There is no "Last 7 days" strip; the user asked for it to be removed.

9. Item tiles (day editor)

9.1 Toolbar

On the left, "**4** of 13 selected" and a "✓ Saved" indicator that appears for 1.4s after each change. On the right, **Select all** (bordered button, disabled when all 13 are selected) and **Clear all** (ghost button, disabled when none are). Select all saves every id; Clear all saves `[]`, which deletes that day.

9.2 Tile grid

- Tiles sit in a **wrapping flexbox** (not CSS grid) with `justify-content: center`. With 13 items the last row is always short, and this keeps it centered. Tile width is `calc((100% - (cols - 1) × gap) / cols)`, set through `--cols` and `--gap`.
- Columns: **5** by default (including in the day modal), **7** for the Today grid at ≥960px, **3** at ≤560px. The gap is 12px, or 8px on phones.
- Each tile is a `<button class="tile" data-id style="--c:ITEMCOLOR" aria-pressed>` containing a check badge (top right), a 46×46 rounded icon square and the name.

9.3 Tile states (final design, as requested by the user)

State	Look
Unselected	White card (`--surface`) with a 1.5px `--border` outline, 14px radius and a small shadow. The icon square has a light tint of the item color (`color-mix(--c 14%, surface)`); the icon itself is drawn in the item color (lightened 40% toward white in dark mode, using `--lift`).
Hover	Lifts 2px with a medium shadow. The background gets a 7% tint of the item color, the border a 45% mix, and the icon square a 22% tint.
Pressed	Scales to 0.97.
Selected	**Minimal style: the box is NOT filled.** The background stays white. The border becomes a **light green ring, the same for every item**: `--select-ring` = `#8FCB8F` in light mode and `#3F9A3F` in dark, drawn as the border plus a 1px inset box-shadow. A **22px round checkmark badge in primary green #006600 with a white check, the same for every item**, fades and scales in at the top right.
Focus	2px focus-visible outline in `--accent-text` .

10. Calendar view and day modal

10.1 Layout

- Page head: the eyebrow "CALENDAR", an H1 with the month and year ("October 2026"), and a summary line ("26 days logged, 137 items in total", or "Nothing logged this month yet. Select a date to add what you ate."). The head has **no Fewer/More legend**; the user asked for it to be removed.

- Toolbar inside the card: a previous-month arrow, a **month** select (January–December), a **year** select and a next-month arrow, with a **Today** button on the right.
- Year select range: from `min(currentYear − 30, first logged year, shown year)` to `max(currentYear + 10, shown year)`, newest first. The arrows can go past either end; the list is rebuilt to include the shown year.

10.2 Grid

- 7 columns, weekday headers from `weekdayNames(weekStart)`. The grid starts at `startOfWeek(first of month)` and has `ceil((days from start to month end + 1) / 7)` rows, 4–6 weeks. Days from the neighboring months are shown at 42% opacity and can still be clicked.
- Each cell is a button with `data-key` and `data-level = FT.level(count)`. It contains the day number and, if anything is logged, a badge "**5**/13". Its aria-label is the full date, "(today)" when it applies, and "N of 13 items" or "nothing logged".
- **Shading by level** (background): level 0 = surface; 1 = `--heat-1`; 2 = `--heat-2`; 3 and 4 = `--heat-3` / `--heat-4` with white text. Levels 1 to 4 drop the border.
- **Today:** the day number sits in a filled `--ink` circle with `--bg` text. **Selected day:** a 2px `--ink` outline, offset 1px.
- **Clicking a day** selects it and opens the day modal.
- `jumpTo(key, {open})` moves to that month, selects the day, focuses the cell and scrolls it into view, and plays a 1.2s ring "flash" animation. Analytics uses it for "View in calendar".
- The calendar re-renders when the store changes, when settings change (week start) and on `dayfocus`.

10.3 Day modal (`FT.openDay`)

Header: an eyebrow with the relative day ("TODAY", "YESTERDAY", "TOMORROW" or the weekday); an H2 with "October 28, 2026"; previous and next day arrow buttons; and the modal's X close button. The body is a day editor (5 columns). Moving to another day calls `editor.setDate` and emits `dayfocus`, so the calendar follows along. On phones (≤560px) the modal is a bottom sheet: full width, 20px top corners, safe-area padding, max height 92dvh. It closes with X, Escape or a click on the backdrop, and focus returns to whatever opened it.

11. Analytics view

Final contents, top to bottom: page head → date range filter → empty-state banner (only when the range has no data) → 4 summary cards → a row with the bar chart and donut → history table. **The "Trend over time" line chart and the "Daily heatmap" were removed at the user's request**; don't add them back.

11.1 Date range filter

Chip	Range (inclusive, local dates)
Last 7 days	today – 6 → today
Last 30 days (default)	today – 29 → today
This month	1st of this month → today
This year	Jan 1 → today
All time	first logged date → max(last logged date, today); just today if nothing is logged
Custom	shows From/To date inputs (prefilled with the last 30 days); the dates are swapped if From is after To

A muted line under the H1 "Your eating patterns" shows the range: "Sep 30, 2026 to Oct 29, 2026 · 30 days". Changing the range resets the history list to 25 rows.

11.2 Empty state

When the range has no logged days: a card titled "No entries in this range", with a **Log today** button (link to #today) and **Load sample data**.

11.3 Summary cards (4; 2×2 at ≤1000px)

Card	Value	Subtext
Days logged	number of days in the range with ≥1 item	"of N days · P%"
Eaten most	icon (in the item color) + name of the item with the highest count	"24 times · 89% of logged days", plus " · tied with K others" on a tie
Eaten least	item with the lowest count (can be 0)	"2 times", plus a tie note
Current streak	N "days" (overall, not limited to the range)	"Longest: N days", or "Log today to keep it going" when the streak > 0 and nothing is logged today

For ties, items are sorted by count with a stable sort, so the earlier item in `FT.ITEMS` wins. Each card has an uppercase label and an icon in a tinted square (calendar, trend-up, trend-down, flame).

11.4 "Times eaten per item": bar chart

Chart.js horizontal bar (`indexAxis: 'y'`), items sorted from most to least. **Each bar uses its item's color.** borderRadius 6, maxBarThickness 22, container height 360px (380px on phones). Integer ticks on the x axis, item names in bold on the y axis, tooltip " N times".

11.5 "Share of intake": donut

A Chart.js doughnut with a 64% cutout, using only items with count > 0, sorted from largest share to smallest, in their item colors. It has a 2px border in the surface color and hoverOffset 6. In the center is an HTML overlay with the total item count and "items logged". Next to it is a custom HTML legend: all 13 items in the same order, with a color dot, the name and a percentage; items at 0 are dimmed to 45%. With no data, the donut is one grey ring and tooltips are off. On phones the donut stacks above a 2-column legend.

11.6 Chart styling (shared)

`responsive: true, maintainAspectRatio: false`, animation 450ms, legend hidden, and font family Manrope. Colors are read from CSS tokens on every render (`--muted` ticks, `--border` grid, `--ink` labels, `--tooltip-bg` / `--tooltip-fg` tooltips), so the charts follow the theme. Charts are destroyed and redrawn on every render.

11.7 History table

- One row per logged day in the range. Columns: **Date** (bold "Oct 28, 2026" with the weekday underneath), **Items** (rounded chips, each with the item icon in the item color and the name), **Count** (a pill "6/13" shaded by level) and a **View in calendar** → button that jumps to that date with `FT.views.calendar.jumpTo`.
- **Sorting:** click the Date or Count header. The first click sorts descending; clicking the same header again flips the direction. `aria-sort` is set and the arrow icon changes. Count ties are broken by date. Default: date descending.
- **Search** (top right of the card, "Search items or dates"): a case-insensitive match against the date key, the long date text ("thursday, october 29, 2026") and item names. It filters as you type.
- **Pagination:** 25 rows, then a "Show more" button that adds 25. The footer shows "Showing X of Y days".
- Phones (≤560px): every row becomes a card (date on the left, count pill on the right, chips below, the link last). The sort headers stay visible as a compact bar.

12. Settings view

Hash `#preferences`. Title "Preferences and data", with the subtitle "Everything stays in this browser. There is no account and nothing is sent anywhere." Cards sit in 2 columns, or 1 at ≤1000px:

1. **Appearance:** Theme segmented control [System | Light | Dark], each with an icon; Week starts on [Sunday | Monday], which affects the calendar.
2. **Your data:** a stats line ("108 days logged, from Jul 1, 2026 to Oct 29, 2026 · 4.1 KB", or "No entries yet.") and three groups:
   - **Back up:** Export JSON (`food-group-log-YYYY-MM-DD.json`, the entries object pretty-printed), Export CSV (`food-group-log-YYYY-MM-DD.csv`) and Copy JSON (clipboard; if the clipboard is blocked, a modal opens with the JSON already selected in a read-only textarea).
   - **Restore:** Import file (a label for a hidden `<input type=file accept=".json,.csv">`).
   - **Try it out:** Load sample data.
3. **Reset** (danger card): "Reset all data" (danger button, disabled when nothing is logged). It opens a confirm modal, "Reset all data?", which says "This permanently deletes N logged days from this browser. This can't be undone." and has [Cancel] [Delete everything]. On confirm the store is cleared and a toast appears.
4. **Storage format:** shows the localStorage key, a sample of the format and the list of item ids.

12.1 CSV format

```
Export (wide):
date,milk,yogurt,vegetables,fruit,red_meat,chicken,chicken_wings,fish,beans,sugar,bread,complex_ca
rbs,processed_food
                2026-10-28,1,1,1,1,1,0,0,0,0,0,0,1,0
Also imported (long):  date,items
                       2026-10-29,Milk;Fruit;Chicken      (items separated by ; or |, names or
ids)
```

When importing the wide format, a cell counts as selected if it matches `/^(1|true|yes|x|y)$/i`. Headers match through `normalizeItem`, so readable names work as well. Rows with an invalid date are skipped. JSON import accepts the entries object itself, or `{ "entries": {...} }`.

12.2 Import flow

The file is read, parsed by extension or MIME type, and cleaned. If it holds no valid entries, a toast says so. If the user already has data, a modal titled "Import N days?" offers [Cancel] [Replace all] [Merge] (Merge is the default). With no existing data, the import simply replaces. Merge takes the union of items per day. The file input is reset afterwards, so the same file can be imported again.

12.3 Sample data (`FT.loadSampleData`)

A seeded random generator (mulberry32, seed 20261029) fills **only empty days** among the last 120 days. It never touches today, and skips about 14% of days to leave gaps. Each day, each item is included with this probability: milk .5, yogurt .35, vegetables .82, fruit .7, red_meat .22, chicken .38, chicken_wings .14, fish .2, beans .3, sugar .42, bread .55, complex_carbs .5, processed_food .3. It merges the result and shows a toast like "Added example entries to N days".

13. Design system

Every color is a CSS custom property. Bare `:root` holds the light values. The dark values appear twice: in `@media (prefers-color-scheme: dark) { :root:not([data-theme="light"]) {...} }` and in `:root[data-theme="dark"] {...}`, both with `color-scheme: dark`. **Edit both dark blocks together.** Components only use tokens.

Token	Light	Dark	Used for
--bg	#FFFFFF	#001A00	page background
--surface	#FFFFFF	#052405	cards, tiles, modals
--surface-2	#F6FAF6	#082B08	subtle panels, table row hover
--tint / --tint-2	#E6F2E6 / #CFE6CF	#0C360C / #124712	hover backgrounds, chips, pills
--primary	#006600	#006600	buttons, active tab, checkmarks, progress
--primary-hover	#004F00	#007A00	primary button hover
--ink	#001A00	#E9F3E9	main text, headings
--ink-2 / --muted	#2B452B / #586C58	#C4D6C4 / #93AD93	secondary text
--border / --border-strong	#E2ECE2 / #C9DAC9	#113811 / #1C4C1C	card and control borders
--accent-text	#006600	#6CCB6C	green text, links, eyebrows
--nav-bg	#001A00	#000E00	top bar, mobile tab bar
--select-ring	#8FCB8F	#3F9A3F	outline of selected tiles
--heat-0…4			calendar shading and count pills (light: #EEF4EE #CDE7CD #93CB93 #3F9A3F #006600; dark: #0A2E0A #0F4A0F #166516 #1D821D #27A327)
--danger	#B42318	#FF8A80	reset button, danger card
--chart-primary	#006600	#3DA63D	single-color chart fallback
--tooltip-bg / -fg	#001A00 / #FFF	#E9F3E9 / #001A00	chart tooltips, toasts
--lift	0%	40%	how much item colors are lightened for icons in dark mode

Typography

- Manrope; body 15px with line-height 1.5.
- H1: `clamp(1.5rem, 1.2rem + 1.2vw, 2rem)`, weight 800.
- H2: 1.05rem, weight 700. Modal title: 1.3rem, weight 800.

Shape and depth

- Radii: cards 16px, tiles 14px, buttons and inputs 10px, pills 999px.
- Shadows: sm on cards and tiles, md on hover, lg on modals and toasts.
- Transitions of 0.18–0.25s on background, border, color, transform and shadow.

- Eyebrows and labels: 0.75rem, uppercase, 0.07–0.09em letter-spacing.
- Numbers use `font-variant-numeric: tabular-nums`.
- Buttons are 40px tall (34px for btn-sm). Variants: default (bordered), primary, ghost, danger.

Other components: `.seg` segmented control (pressed button filled with primary); `.chip` range filter pills (pressed = filled primary); `.pill` (tint background, green text); `.card` (16px radius, 1px border, 24px padding, or 18px on phones); `.select-wrap` custom select with a CSS chevron; `.search` input with an icon.

14. Responsive behavior

Breakpoint	Changes
≥ 960px	Today tile grid has 7 columns.
≤ 1000px	Summary cards go to 2×2; the bar chart and donut stack; settings cards go to 1 column.
≤ 760px	Tabs move to a **fixed bottom tab bar** (4 equal columns, icon above label; the active icon sits in a green pill; safe-area bottom padding; `--tabbar-h: 64px`, which pads the page and lifts toasts). The top bar is 56px tall and shows the brand and theme toggle. Cards have 18px padding.
≤ 560px	Tiles: 3 columns, 8px gap, 96px minimum height, 40px icon square, smaller check badge. Calendar: the arrows and selects fill one row and Today wraps below it on the right; cells use aspect-ratio 1/1.12 with smaller numbers; the badge shows only the count (the "/13" is hidden). The modal becomes a bottom sheet. The history table becomes cards, and the donut legend becomes 2 columns.

The page never scrolls horizontally at 375px; this was tested. The side gutter is `max(16px, env(safe-area-inset-left/right))`. `body::before` paints the safe-area band at the top in the nav color. The viewport meta includes `viewport-fit=cover`.

15. Accessibility

- Tiles are real buttons with `aria-pressed`, grouped with `role="group" aria-label="Food groups"`.
- Calendar cells have descriptive aria-labels; icon-only buttons have aria-labels; the sort headers set `aria-sort`.
- Modals have `role="dialog" aria-modal` and a label, keep Tab focus inside, close on Escape, and return focus when closed.
- Toasts and the "Saved" indicator are in aria-live regions. `.sr-only` labels are used for selects and search.
- `:focus-visible` shows a 2px outline. `prefers-reduced-motion` turns animations and transitions off.

16. Decisions made with the user (change log)

These are deliberate product decisions. Keep them unless the user asks otherwise.

#	Request	What was done
1	Original brief: 12 items, calendar, analytics (including line chart and heatmap), settings, the palette above, localStorage, the given format	Built as plain JS for the reasons in section 2.
2	Remove the "Last 7 days" section	The 7-day strip under Today and its CSS were removed.
3	Add Chicken Wings	New item `chicken_wings` between Chicken and Fish, with a custom wing icon. 13 items in total.
4	Replace Simple Carbs with Bread	New item `bread` with a custom bread-slice icon; the old id `simple_carbs` loads as bread.
5	Each item's color should match its color in the Share of intake chart	One color per item in `items.js`, used for the tile icon, bars, donut, chips and the stat card icon.
6	Remove "Fewer – More" from the calendar page corner	The legend was removed from the calendar header.
7	Selection should be modern and minimal, with a checkmark and no filled box	Selected tiles stay white and show a check badge.
8	Remove the Daily heatmap and Trend over time	Both sections, their code, state and CSS were removed from Analytics.
9	All checkmarks green and identical	The check badge is always `--primary` #006600 with a white check.
10	The outline on selected tiles should be light green, the same for all	Added the `--select-ring` token (#8FCB8F light, #3F9A3F dark).

17. Testing checklist

There are no automated tests. These checks were done by hand in a browser (and with scripts run in the page's console) and should be repeated after any change:

1. No console errors on any of the four tabs, in light and dark mode.
2. Tapping a tile toggles it, "Saved" appears, the counter updates and the data survives a reload (`localStorage.getItem('foodGroupLog.entries')`).
3. Select all / Clear all, and their disabled states.
4. Calendar: arrows across a year boundary, the dropdowns, Today, shading and badges, the today and selected styles, and the modal's previous/next day keeping the calendar in step.
5. Analytics: every range including custom; the cards; bars and donut in item colors; history search, sort both ways, Show more, and View in calendar.
6. Settings: export CSV, then import it with "Replace all"; the data should be identical (the key order may differ). Import a long-format CSV with Merge. In the reset dialog, Cancel keeps the data and confirm deletes it.
7. Week start Monday changes the calendar headers.

8. At a 375px width: the bottom tab bar, 3-column tiles, the calendar fitting on screen, the bottom-sheet modal, the history cards, and no horizontal scroll (`document.documentElement.scrollWidth === innerWidth`).

18. Deploying

- **Netlify:** drag the folder onto app.netlify.com/drop, or connect the repo with no build command and publish directory `.`
- **Vercel:** `vercel deploy` with the framework preset "Other" and no build command.
- **GitHub Pages:** push the folder and deploy from a branch, root folder.
- **Single-file build (optional):** a small Python script made the claude.ai preview by taking the text between the `BUILD:` markers in index.html, inlining `css/styles.css` into a `<style>` tag and every `js/...` script into an inline `<script>`, and keeping the theme pre-paint script. The script isn't in the project folder; rewrite it from this description if needed.
- Live preview: https://claude.ai/artifact/FQk68Jq925iudiU6y38NK6 (private to the owner unless shared). Export downloads are blocked in that preview; Copy JSON works.

19. How to make common changes

Add, remove or rename a food item

1. Edit `FT.ITEMS` in `js/items.js` (id in snake_case, name, icon key, color). The position in the array sets the display order.
2. If the icon is new, add its path to the dictionary in `js/icons.js` (24×24, stroke only).
3. Add the id's probability to `odds` in `FT.loadSampleData` (settings.js).
4. If you rename or remove an id, add an alias in `normalizeItem` so saved data still loads. Without one, the old id is silently dropped.
5. Update the item lists in README.md. Counts, badges, CSV columns and charts adapt automatically.

Change colors or the theme

Change tokens in `:root` and in **both** dark blocks of styles.css. Item colors live only in `items.js`. Keep item colors dark enough to read on white, and remember dark mode lightens icons by `--lift`.

Add a view

Add a `<section class="view" id="view-NAME" data-view="NAME" hidden>` and a tab link to index.html; create `js/views/NAME.js` that defines `FT.views.NAME = { show(el) }`; add NAME to `VIEWS` in app.js; add the script tag before app.js. The mobile tab bar uses `repeat(4, 1fr)`, so change that too.

Change the calendar shading thresholds

Edit `FT.level` in calendar.js. It's also used by the count pills in the history table.

20. Known limitations

- Charts need internet access for the Chart.js CDN. For fully offline use, download `chart.umd.js` into the project and change the script src.
- Data lives in one browser on one device. Moving it relies on export and import.
- Dates and labels are always US English (en-US), whatever the browser locale.

- History pagination keeps everything in memory; that's fine for years of daily data.
- There are no automated tests (see section 17 for the manual checklist).